

5.1 Funciones y operaciones

5.1.1. Espacio de direccionamiento: ¿Qué limitación física y económica define el tamaño práctico del espacio de direccionamiento en un procesador moderno de 64 bits frente a su límite teórico?

R: dado a que los sistemas entre 1 y 256 TB no es posible direccionar 2^{64} bytes debido a que no se tiene esa cantidad de almacenamiento disponible, además de tenerlo el costo económico es demasiado alto.

5.1.2. MMU: ¿Cuál es la función específica de la Unidad de Manejo de Memoria (MMU) y cómo intervienen los registros base y límite para garantizar la seguridad entre procesos?

R: la MMU fue introducida debido a que en los sistemas modernos se requieren cargar múltiples procesos en memoria. Este hardware se encarga también de gestionar que ningún proceso pueda escribir en el espacio de memoria de otro proceso. Para aplicar esto último, la MMU utiliza dos registros, el base y el límite para determinar el comienzo y el final de la sección en memoria de cada proceso.

5.1.3. La memoria caché: Explicá los tres principios de "localidad de referencia" (temporal, espacial y secuencial) que hacen que la caché sea efectiva.

R: la localidad de referencia consiste en que las páginas de memoria no se acceden aleatoriamente a la memoria, sino que respetan un orden. Para eso se definen, dentro de la localidad de referencia, tres principios:

- Localidad temporal: un recurso empleado recientemente puede volver a emplearse en un futuro cercano.
- Localidad espacial: un recurso que aún no fue requerido es probable que sea requerido en un futuro cercano si algún recurso cerca de él fue requerido.
- Localidad secuencial: la memoria tiende a ser requerida de forma secuencial.

5.1.4. El espacio de memoria de un proceso: Describí cómo se dividen lógicamente las cuatro secciones de un proceso en memoria (texto, datos, heap, stack) y en qué dirección crecen dinámicamente las últimas dos.

R: al cargarse un proceso en memoria, se proporcionan las estructuras para que este guarde la información relativa a su ejecución. Lo hace en las siguientes secciones:

- Segmento de texto: en este segmento se almacenan las instrucciones a ser ejecutadas, y se encuentran en las direcciones más bajas de la memoria.
- Segmento de datos: en este segmento se almacenan las variables globales y los datos inicializados (no inicializados también). Fijado en tiempo de compilación.

- Segmento Heap: espacio destinado para la asignación de memoria dinámica pedida en tiempo de ejecución. Se encuentra por encima de segmento de datos y sus direcciones crecen hacia arriba.
- Segmento Stack: espacio destinado para almacenar las llamadas a funciones con sus parámetros, direcciones de retorno, variables locales. Ocupa la parte más alta de la memoria y sus direcciones crecen hacia abajo.

5.1.5. Resolución de direcciones: ¿Cuáles son los tres momentos distintos en los que se pueden traducir (resolver) las referencias de memoria de un programa desde su compilación hasta su uso real?

R: el compilador convierte el programa en lenguaje máquina y luego lo sustituye por la dirección de memoria donde se encuentra la variable o la función. Existen diferentes estrategias para que los programas sean traducidos:

- En tiempo de compilación: el texto del programa tiene la dirección absoluta de las variables y funciones.
- En tiempo de carga: el cargador actualiza las referencias a memoria dentro del texto para que apunten al lugar correcto antes de cargarse en memoria y de iniciar su ejecución.
- En tiempo de ejecución: el sistema no hace referencia a una ubicación exacta en la memoria, sino que lo hace en con una base y desplazamiento, permitiendo que sea reubicado en la memoria si está siendo ejecutado, y sin sufrir cambios.

5.2 Asignación de memoria contigua

5.2.1. Partición de la memoria: En un sistema de asignación contigua, ¿cómo visualiza el sistema operativo la memoria libre al inicio y cómo la va subdividiendo?

R: cuando el sistema operativo se inicia, toda la memoria se ve como un único bloque libre y la subdivide para satisfacer sus registros. Esto lo realiza asignando a cada programa a un bloque contiguo de memoria de tamaño fijo.

5.2.2. Fragmentación: Diferenciá claramente el concepto de fragmentación interna del de fragmentación externa, dando un ejemplo de cómo ocurre cada una.

R: tenemos dos tipos de fragmentación, la externa y la interna. La externa consiste en que los procesos se van cargando en el sistema operativo por bloques y a medida que se van liberando se van generando bloques libres dispersos en la memoria, dejando memoria libre por todo el espacio de direcciones. Para ejemplificar, supongamos que tenemos 4 procesos cargados, P1 (4 KB), P2(2 KB), P3 (4 KB), P4 (2 KB), el proceso P2 y P4 son liberados, llega el proceso P5 (4KB) y este no puede ser cargado en la memoria debido a que tenemos dos bloques de 2 KB libres. Aunque el tamaño total de los bloques sea igual al tamaño del proceso, como están dispersos en la memoria, no podemos cargar el proceso. La fragmentación interna consiste en la memoria

que se pierde dentro de un bloque debido a que no se utilizará. Por ejemplo, si se manejan 8 bytes en un bloque y un proceso requiere 2, tenemos 6 bytes que se pierden dentro del bloque y nunca se utilizarán.

5.2.3. Compactación: ¿En qué consiste la compactación para resolver la fragmentación externa y por qué su uso implica una penalización severa en el rendimiento del sistema?

R: la compactación consiste en agrupar los bloques de memoria de forma contigua así juntar los espacios libres de memoria. Conlleva un costo elevado porque requiere copiar los datos y pegarlos en lugares diferentes.

5.2.4. Intercambio (swap): Cuando la memoria se agota bajo este esquema, ¿qué pasos sigue el sistema operativo para realizar el intercambio (swap) de un proceso hacia el almacenamiento secundario?

R: el sistema operativo para realizar swap con el almacenamiento secundario primero selecciona que un proceso que no se esté ejecutando, luego copia la imagen completa en el disco, una vez que el proceso sea requerido, vuelve a cargarse en la RAM. Todo esto conlleva un tiempo determinado por la velocidad del disco.

5.3 Segmentación

5.3. / 5.3.1. Permisos: ¿Cómo se compone una dirección lógica en segmentación y qué ventajas de seguridad aporta el poder definir permisos independientes (lectura, escritura, ejecución) por cada segmento?

R: las direcciones en segmentación se organizan mediante direcciones lógicas, en lugar de direcciones físicas, compuestas por un identificador de segmento y un desplazamiento. La MMU es la encargada de traducir las direcciones lógicas a físicas. La segmentación proporciona la característica de que a cada segmento se le puede dar diferentes permisos, ya sea de escritura, lectura o ejecución y, por lo tanto, un proceso asignado a determinado de segmento deberá cumplir con esos permisos. Nos da ventajas de seguridad como pueden ser que datos corruptos o maliciosos no puedan modifiquen un segmento, también, al restringir los permisos de cada segmento, los códigos maliciosos no podrán ingresar a segmentos de ejecución para alcanzar su objetivo. Por último, se restringe el acceso a estructuras del sistema evitando que procesos no privilegiados modifiquen memoria.

5.3.2. Rendimiento: ¿Por qué hacer *swap* en un sistema segmentado es mucho más eficiente y selectivo que hacerlo en un sistema de asignación contigua pura?

R: la segmentación permite realizar un intercambio selectivo y organizado en base a que, en lugar de mover todo el espacio de memoria de un proceso, solo se intercambian los segmentos activos. Además, el sistema puede usar políticas de reemplazo de páginas (como LRU) para decidir cuál de ellas intercambiar. Si tenemos segmentos de lectura, el intercambio no se realiza si ya están cargados en disco, marca el segmento como no presente, y cuando lo necesita

nuevamente lo activa. Si una biblioteca o segmento de código ya existe en disco, lo puede mapear desde allí, sin necesidad de primero cargarlo en RAM.

5.4 Paginación

5.4. / 5.4.1. Tamaño de la página: ¿Cómo soluciona la paginación la necesidad de la compactación, y qué *trade-off* (compromiso) existe a la hora de elegir un tamaño de página muy grande frente a uno muy chico?

R: la paginación soluciona el problema de la compactación dividiendo la memoria física y el espacio de direcciones en páginas lógicas y marcos. Las direcciones lógicas se dividen en el número de página y un desplazamiento que indica la posición exacta dentro de la página. Al elegir un tamaño de página pequeño, la cantidad de entradas a la tabla de páginas aumentaría generando un aumento en el tamaño del PCB debido al almacenamiento de la tabla de páginas del proceso.

5.4.2. Almacenamiento de la tabla de páginas: ¿Por qué los sistemas modernos abandonaron el uso de registros dedicados para almacenar las tablas de páginas enteras y pasaron a usar registros como PTBR y PTLR apuntando a la RAM?

R: los sistemas actuales abandonaron el uso de almacenar la tabla de páginas enteras en registros debido a que en los sistemas antiguos el tamaño de esta era muy chico a diferencia de la actualidad. Por ejemplo, antes, en arquitecturas de 16 bits con tamaño de página 8 KB, almacenaban solo 8 entradas, por lo que el tamaño era menor. Ahora en arquitecturas de 32 o 64 bits con tamaño de páginas 4 KB, se guardan megabytes impidiendo el almacenamiento en registros. Se utilizan registros, PTBR para almacenar el comienzo de la tabla y PTLR para almacenar el tamaño de la tabla.

5.4.3. La TLB: Explicá qué es el Translation Lookaside Buffer (TLB), cómo mitiga el doble acceso a memoria RAM, y qué sucede durante un "TLB miss".

R: la TLB es una tabla asociativa que funciona como un caché en la MMU almacenando las páginas recientemente usadas. Al requerir una página, primero se revisa si esta se encuentra en la TLB, si está presente, en un ciclo de reloj, se obtiene la dirección física de la página, en caso de no encontrarse en la TLB, se accede a la tabla de páginas para buscarla allí, donde luego, la página se carga en la TLB.

5.4.4. Paginación multinivel / hasheo: ¿De qué manera dividir la tabla de páginas en múltiples jerarquías reduce el consumo masivo de memoria RAM que exigen los procesos pequeños?

R: dividir la tabla de páginas en varios niveles reduce el tamaño dividiendo el identificador de páginas en múltiples niveles. Si tenemos dos tablas, y una arquitectura de 32 bits, los primeros 10 bits corresponden al directorio de páginas, los segundos 10 bits corresponden a la segunda tabla de páginas y los 12 bits restantes al desplazamiento. Esto reduce la memoria ya que solo se cargan en RAM las tablas correspondientes a regiones de memoria utilizadas.

5.4.5. Memoria compartida y Copy on Write: Desarrollá el mecanismo de "Copy on Write" que ocurre al llamar a fork(): ¿qué comparten padre e hijo inicialmente y qué evento dispara la creación de un nuevo marco físico?

R: el mecanismo "Copy on Write" consiste en que al realizar un fork, se genera una tabla de página nueva para el proceso hijo, pero esta apunta a los mismos marcos, marcados como lectura, evitando la copia innecesaria de páginas. A medida que un proceso quiera escribir una página, esta se copia en un nuevo marco y esta apunta al proceso que la modificó.

5.5 Memoria virtual

5.5. / 5.5.1. Paginación sobre demanda: ¿Qué es un fallo de página y cuál es la secuencia exacta de acciones que toma el sistema operativo de forma "lazy" para resolverlo?

R: en la memoria virtual cada proceso observa tener una idealización de la memoria creyendo tener acceso a todo el espacio de direccionamiento. En la programación bajo demanda, por hardware, se agrega un bit de validez para verificar si esa página se encuentra en memoria. Si esta no está marcada (bit en 0) se genera un fallo de página y así el sistema deberá traerla a memoria. Es lazy porque carga en memoria las páginas a medida que se generen los fallos de página. El acceso que realiza el sistema es el siguiente:

1. Verifica en el PCB si se encuentra la página cargada en el proceso.
2. Si no es así, el sistema termina con su ejecución.
3. Realiza el swap, buscando en la tabla un marco libre.
4. Solicita al disco la lectura de la página hacia el marco.
5. Una vez terminada la lectura, se actualiza el PCB y la TLB.
6. El proceso reanuda desde la instrucción que generó el fallo de página.

5.5.2. Reemplazo de páginas: ¿Para qué sirve el "dirty bit" (bit de modificación) y cómo ayudan los algoritmos como LRU o el Reloj/Segunda Oportunidad a decidir qué página expulsar?

R: el dirty bit, agregado por hardware, disminuye la probabilidad de tener que realizar doble transferencia. Este bit se indica, si está encendido apagado, que en el disco se encuentra una copia de la página idéntica a la copia en memoria, si está encendido, se deberá grabarla en disco. El algoritmo de reemplazo de página LRU decide tomar como página víctima la menos recientemente utilizada, es decir la que hace más tiempo no fue empleada. Por otro lado el algoritmo de Reloj/Segunda oportunidad busca encontrar una página víctima en base a la que tiene su bit de referencia en 0, es decir, que no ha sido empleada recientemente. Cuando una página es cargada en la tabla, esta comienza con su bit de referencia encendido, cuando ocurre un fallo de página, un cabezal busca una página que tenga su bit apagado, si ninguna tiene apagado su bit, se le da una segunda oportunidad apagándolo y pasando a la siguiente. Esto hasta encontrar una que cumpla con lo dicho antes y reemplazarla.

5.5.3. Asignación de marcos: Contrasta el esquema de "reemplazo local" con el de "reemplazo global", mencionando la rigidez de uno frente a la inconsistencia de rendimiento del otro.

R: el reemplazo de marcos puede darse de diferentes formas, el reemplazo local consiste que, ante un fallo de página, el reemplazo se realiza con páginas del mismo proceso que causó el fallo. Mientras que, en el reemplazo global, se puede reemplazar cualquier página de la tabla, sea del mismo proceso o no.

5.5.4. Hyperpaginación: ¿Cómo se define el estado de hyperpaginación (thrashing) y cómo el concepto de "conjunto activo" ayuda al sistema operativo a decidir suspender procesos para solucionarlo?

R: la hyperpaginación consiste en que el sistema, ante muchos fallos de página, pasa más tiempo realizando swap que ejecutando procesos. Puede surgir en el contexto de reemplazo local donde el proceso tiene pocas páginas, y un número elevado de fallos. El concepto de conjunto activo consiste en el conjunto de páginas cargadas en memoria que está usando un proceso. El sistema tendría que mantener cargado el conjunto activo de un proceso, y si no es posible, debería suspender temporalmente ese proceso.

5.6 Memoria virtual en Linux

5.6.1. Zonas de memoria: ¿Por qué el núcleo de Linux reserva bloques específicos de direcciones físicas y no permite que toda la RAM se asigne libremente a la memoria dinámica?

R: hay bloques reservados en la memoria debido a que estos se usan para el hardware, por ejemplo, los primeros 4 KB se utilizan para la configuración del hardware durante el POST, como también luego se reserva memoria para la configuración de la BIOS y parte del núcleo.

5.6.2. Optimizaciones: ¿En qué consiste el "mapeo de archivos en memoria" y cómo la "lectura por adelantado" optimiza el rendimiento general del disco?

R: consiste en que un proceso puede solicitar una porción de memoria para se refiera a un archivo, por lo tanto, leer/escribir en un archivo consiste en leer/escribir una dirección de memoria. Tenemos que entonces no es conveniente escribirla en swap, sino que es conveniente escribirla en el espacio reservado para el archivo. La lectura por adelantado consiste en que, si leemos un bloque, es muy probable que sea necesario leer en poco tiempo un bloque cercano a este, por lo que se agregan en caché para evitar que se deba esperar a leer la solicitud debido a que si esperamos deberemos posicionar nuevamente para buscar la solicitud.

5.6.3. NUMA: ¿Qué implica la arquitectura de Acceso No Uniforme a Memoria (NUMA) en equipos multiprocesadores para la gestión de memoria de Linux?

R: implica que muchas porciones de las porciones de memoria están asociadas a un procesador, brindando mayor velocidad de acceso a este. La memoria es

particionada en nodos y puede subdividirse en zonas, por lo que las estructuras de gestión de memoria deben replicarse para cada nodo o zona.

5.6.4. Reverse mapping: ¿Cuál es la utilidad del "reverse mapping" cuando el algoritmo de reemplazo necesita evaluar un marco candidato para ser liberado?

R: el reverse mapping consiste en identificar que entradas de la tabla de paginación apuntan a un determinado marco para obtener información sobre un proceso candidato a ser reemplazado, es decir, podemos obtener si está bloqueado, que procesos lo usan, si fue accedido recientemente.

5.6.5. Otras reservas: ¿Para qué situaciones críticas mantiene el núcleo de Linux un *pool* reservado de "ubicación atómica"?

R: se utiliza la ubicación atómica cuando el procesamiento de código se realiza en una sección crítica del núcleo. Entonces se destina una reserva de páginas para estos casos.

5.7 Ubicación de memoria y gestión de memoria libre

5.7. / 5.7.1. Buddy System y Slab Allocator: Explicá cómo el "Buddy System" gestiona la memoria libre dividiendo y fusionando bloques en potencias de 2, y por qué necesita complementarse con el "Slab Allocator" para pedidos más chicos.

R: el buddy system se encarga de organizar la memoria en bloques de tamaño potencias de dos, donde la información sobre los bloques se guarda en una estructura de datos similar a un arreglo de listas. Cuando un proceso solicita memoria, se busca el bloque más pequeño que pueda cumplir con el pedido. Si solo hay bloques más grandes, se dividen y el sobrante se coloca en la lista correspondiente. Si hay bloques colegas (adyacentes y alineados), al liberar memoria se fusionan, reduciendo la fragmentación externa. El slab allocator es utilizado cuando queremos reservar porciones mas chicas de memoria, permitiendo reducir la fragmentación interna.

5.8 Reclamo de páginas

5.8.1. Tipos de marcos / PFRA: Según los principios del PFRA de Linux, ¿qué tipos de páginas se prioriza liberar primero (ej. cachés vs anónimas) y por qué es preferible elegir páginas "limpias"?

R: se liberan primero las páginas que provoquen menos daño, estas son las páginas de la cache (no se necesita acceder a disco). Es preferible elegir páginas limpias debido a que ahorra la escritura en el disco.

5.8.3. Cuando la memoria escasea: Si el reclamo estándar (kswapd) no alcanza, ¿qué medidas agresivas toma el sistema operativo antes de invocar al "Out of Memory Killer", y a quién busca eliminar este último?

R: para recuperar la memoria el sistema puede achicar el tamaño de los cachés. Si se complica la recuperación de memoria, el Out of Memory Killer, el núcleo

puede decidir eliminar el proceso. Se intenta elegir un proceso grande y que no esté involucrado a secciones críticas, o en tareas de entrada/salida o del núcleo.

5.8.4. Consideraciones sobre swap: Describí el riesgo de concurrencia conocido como "Multiple swap-in" que puede ocurrir si el caché de swap no se gestiona correctamente.

R: el múltiple swap-in consiste en que dos procesos intentan acceder simultáneamente a una misma página que está en swap. Si ambos inician la lectura sin coordinación, cada uno terminará con una copia separada de la página.